UNIVERSITY OF ELOUED

Faculty of Exact Sciences
Department of Computer Science

TP02: HOW TO WORK WITH BIG DATA FILES (+5GB)

Comparing Standard Pandas, Pandas Chunking, and Dask DataFrames

Second-year Master: AI & Data Science
Academic Year: 2026/2027
Module: Big Data Analytics

Student: Abderrazzak Karoui

1. Dataset and Environment

The notebook downloads the dataset “steam-reviews-dataset” from Kaggle using kagglehub.d("forgemaster/steam-reviews-dataset"). The downloaded directory contains **11 CSV files** with a combined reported size of **5.32 GB.**

Dataset property	Observed value
Number of CSV files	11
Combined file size	5.32 GB
Rows loaded by Standard Pandas	15,437,471
Columns	13
Dask partitions	11
Chunk size	100,000 rows

13 columns :[steamid, appid, voted_up, votes_up, votes_funny, weighted_vote_score, playtime_forever, playtime_at_review, num_games_owned, num_reviews, review, unix_timestamp_created, and unix_timestamp_updated.]

2. Methodology and Implementation

2.1 Standard Pandas loading

- The first method reads each CSV file using pd.read_csv(), stores the resulting DataFrames in a list, and combines them with pd.concat(..., ignore_index=True)..
- No MemoryError was recorded. but expected OOM and handle it with a message,

2.2 Pandas chunking

- The second method uses pd.read_csv(file, chunksize=100_000), processing one chunk at a time for every CSV. For each chunk, it selects columns with dtype “number” and excludes [steamed, appid].
- .the means was calculated by total_sum / total_count.

2.3 Dask DataFrame

- The third method creates a Dask DataFrame using dd.read_csv(..., blocksize=None, assume_missing=True). The notebook reports 11 partitions, corresponding to the 11 CSV files in this run. Dask initially constructs a lazy representation of the data and operations rather than immediately materializing every result as a single Pandas DataFrame.
- The code selects numeric columns, exhausts [steamid and appid] and builds sum, count, and row-count tasks. It passes all three tasks to dd.compute(...) together. This lets Dask coordinate their execution and apply optimization and scheduling behavior..

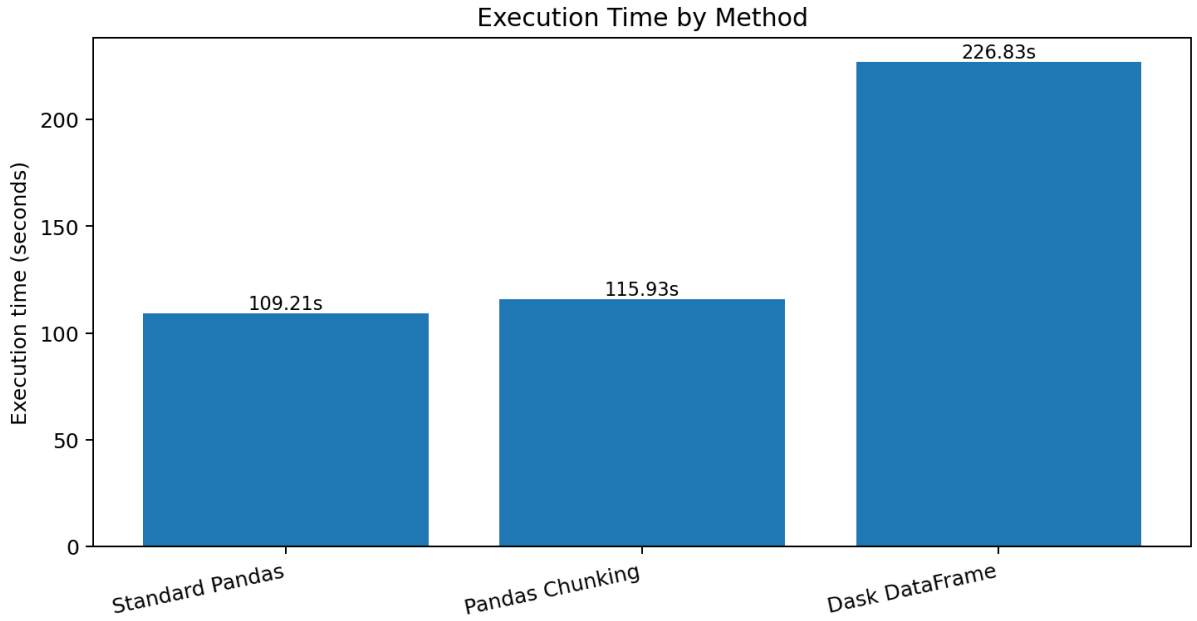
3. Experimental Results

These are results from this specific run on colab. Times are in seconds and memory values are in GB.

Results could changes based on power of Cpu or Gpu and if its Tpu that would be more better

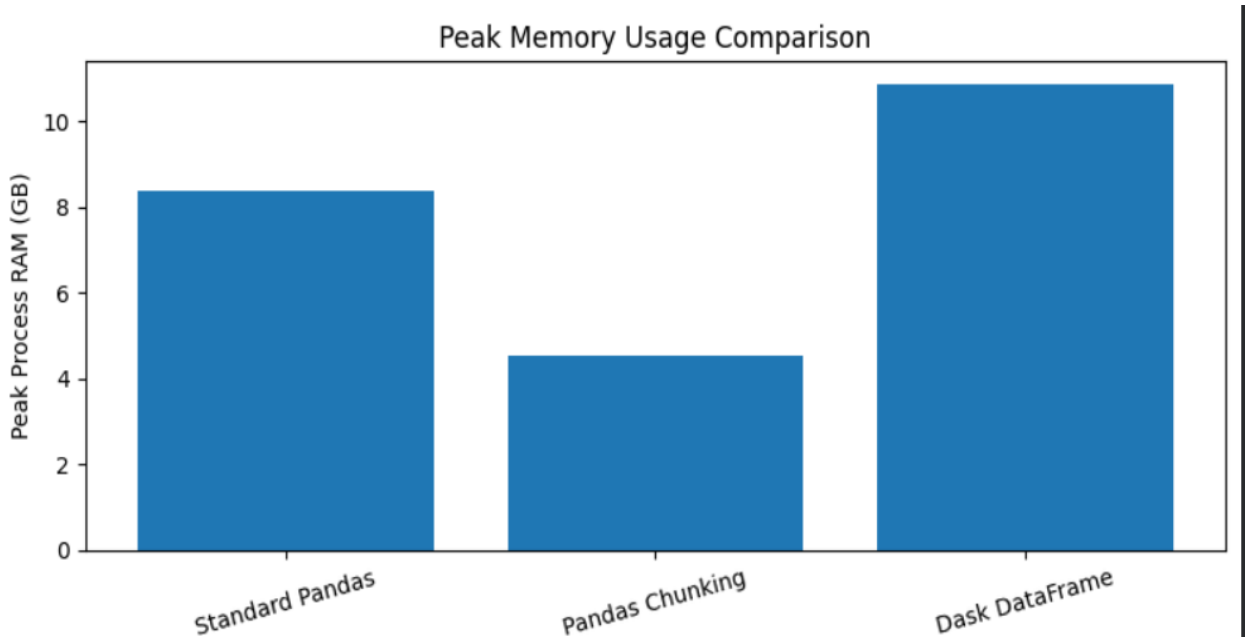
Method	Execution time (s)	Maximum sampled RSS (GB)
Standard Pandas	109.21	8.39
Pandas Chunking	115.93	4.55
Dask DataFrame	226.83	10.84

3.1 Execution-time comparison



Standard Pandas completed in **109.21 seconds**, Pandas Chunking in 115.93 seconds, and Dask in **226.83 seconds**. Chunking took **6.72 seconds** longer than Standard Pandas (about **6.2% longer**). Dask took about 2.08 times as long as Standard Pandas in this run. These are results from this specific run, not universal performance rankings..

3.2 Memoery comparison



Standard Pandas reached **8.39GB** , Pandas Chunking **4.55GB**, and Dask **10.84GB**. standar pandas reach up to 8.39GB longer than Chunking (about **184.39% longer**). Dask took about **1.293 times** as long as Standard Pandas in this run, by the way. These are results from this specific run, not universal performance rankings

4.Conclusion

This practical work processed a **5.32 GB** dataset containing **15,437,471** rows across 11 CSV files. Standard Pandas loaded and concatenated the files successfully in **109.21 seconds**, with a maximum Memory of **8.39 GB**. Pandas chunking processed the same number of rows in **115.93 seconds** and recorded a maximum sampled RSS of **4.55 GB**. Dask used 11 partitions and computed row counts, sums, and counts together, but took **226.83 seconds** and reached a maximum sampled RSS of **10.84 GB**.

For this dataset and execution environment, Standard Pandas was fastest, while chunking had the lowest recorded maximum process memory. Dask's lazy task-based design remains useful for scalable workflows, but the observed results show that it should be benchmarked rather than assumed to be faster or more memory-efficient. Future testing could repeat each method, align the numeric-column selection, monitor memory outside the notebook process, and compare the Dask task graph before and after explicit optimization.